Review: data structures & algorithms
Select or write the most accurate answer for each question.

When you're finished, click "Send for review" at the top to submit.

1.

What value is stored in matrix[1][2] immediately after the assignment line runs?

```
matrix = [
    [2, 4, 6],
    [1, 3, 5],
    [10, 20, 30]
]
matrix[1][2] = matrix[0][1] + matrix[2][0]
```

8

X 14

12

3

2.

What does items look like after this code runs?

```
items = [1, 2, 3]
items.append(4)
items[0] = items[-1] + items[1]
```

[1, 2, 3, 4, 6]

[4, 2, 3, 4]

X [6, 2, 3, 4]

[5, 2, 3, 4]

3.

A list has 4 elements. Which of the following will raise an IndexError?

X items[4]

items[3]

items[-1]

items[-4]

4.

What is the final value of grid[0][2] after this code runs?

```
grid = [
    [1, 2, 3],
    [4, 5, 6]
]
grid[0][2] = grid[1][0] + grid[0][1]
```

3

5

X 6

7

5.

A stack starts empty. These operations run in order:

PUSH "A"
PUSH "B"
POP
PUSH "C"
PUSH "D"
POP
What is on top of the stack at the end?

"A"

"B"

X "C"

"D"

6.

Which code correctly implements a queue that processes items in arrival order using deque?

```
from collections import deque
jobs = deque()
jobs.append("job1")
jobs.append("job2")
jobs.append("job3")
next_job = ???
```

```
next_job = jobs.append("job4")
```

```
next_job = jobs[-1]
```

```
next_job = jobs.pop()
```

X
```
next_job = jobs.popleft()
```

7.

A web browser keeps a history of visited pages so the user can press Back to return to the previous page. Which principle and operation pair is correct?

X LIFO — pop the most recently added page

FIFO — dequeue the oldest page

LIFO — pop the oldest page

FIFO — dequeue the most recently added page

8.

Starting with an empty queue, these operations run:

ENQUEUE "X"
ENQUEUE "Y"
DEQUEUE
ENQUEUE "Z"
DEQUEUE
What is the single remaining item in the queue?

"X"

"Y"

Nothing — the queue is empty

X "Z"

9.

An algorithm runs a single loop over a list of n items. Inside the loop it applies 3 operations on each element. What is its Big O time complexity?

XX O(3) WRoNG

O(n) XXX<Correct

O(n²)

O(3n)

10.

What is the Big O complexity of this function?

```
def find_pair(items):
    for i in range(len(items)):
        for j in range(len(items)):
            if items[i] + items[j] == 0:
                return True
    return False
```

X O(n²)

O(n)

O(1)

O(2n)

11.

An algorithm does $4n^2 + 8n + 15$ operations. Which Big O expression correctly classifies it?

O($4n^2$)

O(8n)

O(n)

X O(n^2)

12.

A teammate says: "Algorithm A finishes in 2 seconds on my laptop, so it must be O(1)."

What is wrong with this reasoning?

Nothing — 2 seconds is fast, which proves constant time

O(1) only applies to algorithms written in Python

X Big O describes how steps grow with input size, not how fast an algorithm is

The algorithm must be O(n) if it finishes in under 5 seconds

13.

You need to find a name in an unsorted list of 50 contacts. Which search approach is correct here?

Binary search — it is always faster than linear search

X Linear search — binary search requires a sorted list and this one is not sorted

Binary search — 50 items is large enough to justify it

Either works identically on unsorted data

14.

Bubble sort is applied to [3, 1, 4, 2]. Write the list after the first full pass.

[1, 3, 2, 4]

15.

A program needs to look up product prices by product ID many times as the number of products grows.

```
prices_list = [("A10", 12.99), ("B20", 9.50), ("C30", 18.25)]
prices_dict = {"A10": 12.99, "B20": 9.50, "C30": 18.25}

target_id = "B20"
```

Which approach is more efficient for repeated lookups by target_id, and why?

Use the list of (product_id, price) pairs because each pair keeps the ID and price together, so lookup by product ID is automatically O(1).

X Use the dictionary because the product ID can be used as a key, giving average O(1) lookup instead of checking pairs one by one.

Sort the list of pairs before every lookup because sorted data makes a single price retrieval constant time.

Append the target product ID to the list before searching because adding the target reduces the number of comparisons needed.

16.

Insertion sort processes [5, 2, 8, 1]. After inserting the second element into its correct position within the sorted portion, what does the list look like?

[2, 5, 8, 1] XXXX Correct

X [1, 2, 5, 8] WRoNG

[5, 8, 2, 1]

[2, 5, 1, 8]

17.

A teammate wants to remove every failing score from a list. They write this code:

```
scores = [82, 55, 40, 70, 91]

for score in scores:
    if score < 60:
        scores.remove(score)
```

Explain why removing elements from scores while looping over scores is fragile. What

kind of result can this cause, and how could the code be made more reliable?

You will get a ValueError if you try to remove an element no longer in tho list. The error possibility could be fixed by creating a new list and if score > or equal to 60 then append score to new list

Incorrct -Q17: 3/5 – You pointed out that creating a new list avoids problems, but the chief issue is skipped elements due to index shifting, not a ValueError.

18.

A developer builds a task manager where completed tasks are stored in a list. When a user clicks 'Undo Last', the most recent task should be un-completed. When a user clicks 'Next Pending', the oldest pending task should be shown first.

Explain which data structure principle applies to each feature and what would go wrong for the user if both features used the same principle.

Undo last is a stack operation, next pending is a queue operation. If your dequeiing with popkeft() the stack undue operation could cause an error trying to pop from an empty list or vice versa

Incorrect - Q18: 3/5 – Correctly matched stack to Undo and queue to Next Pending. Needed to note that using the same principle would reverse the intended order, not primarily raise an empty-structure error.

19.

Two algorithms solve the same problem. For an input of size 10, Algorithm A takes 20 steps and Algorithm B takes 100 steps. A developer concludes: "Algorithm A is always better."

Challenge this conclusion. What information is missing, and describe a situation where Algorithm B could be the more practical choice.

Yeah if the data set grows to a large number that's correct, however if the data is small the difference won't be very noticeable. However if the longer running algo is easier to understand and read, It may be better then a confusing bit of faster code that's not really necessary

Almost Correct - Q19: 4/5 – Nicely noted that growth rate info is missing and small inputs can be deceptive.
 A clearer example contrasting Big-O classes would have earned full points.

20.

A developer needs to search a list of 500,000 product IDs for a specific entry. The list is already sorted. They argue: "Linear search is simpler to implement, so I'll use that."

Explain what the practical difference would be for a user at this scale, and under what condition the developer's preference for simplicity would be a reasonable trade-off.

A linear search 500,000 items is a huge number that could leave the user waiting an long time vs perhaps a binary search, which works on this sorted list, scales way better with large datasets however it is believed using Pythons built-in sort, which is highly optimized for efficiency and speed is the smartest choice. Especially as the amount of data grows larger. The devs own coded linear search would be fine with smaller amounts of data say a few hundred

Almost - Q20: 4/5 – You highlighted O(log n) versus O(n) impact and when linear search could be acceptable.

Including a rough comparison count (≈19 vs 500 000 checks) would have completed the explanation.